

Homework 21: Build The Framework You Would Otherwise Import

Extend the micro harness until it does the things a framework does, then find the bug that `assert` hides in production.

Zero third-party dependencies. Everything here is JDK only.

Part One: A Better Harness

Create `TestKit.java` in the classic form.

Write a `Suite` class supporting:

- `test(String name, Runnable body)` — records a pass or a failure
- `assertEquals(expected, actual)` — failure detail must show **both** values
- `assertThrows(Class<? extends Throwable>, Runnable)` — passes only if that exact type is thrown

`assertThrows` has **two** failure cases, not one. Handle both, with different messages:

1. Nothing was thrown
2. Something was thrown, but the wrong type

Then run a suite containing at least: two passes, a failing `assertEquals`, both `assertThrows` failure modes, and a test that throws something unexpected.

Add **parameterised** tests: one behaviour checked against four rows of input, generated in a loop. Name each case with its inputs, and say in a comment why naming it `case 3` would be worse.

Print a summary line, and print the exit code a real harness would return.

Part Two: Isolation

Write a small mutable class, then run two checks against it **twice**:

1. Sharing one instance between them
2. Giving each its own

Show the second check failing in the shared version and passing in the fresh one.

Explain It

A comment on why an order-dependent test is **worse than no test at all**. Your answer should say what happens later, to whom, and why the absence of a test is more honest.

Part Three: The Bug `assert` Hides

Write a class with a method containing this:

```
assert items.remove(item) : "expected " + item + " to be present";
```

Run your program **without** `-ea` and print the collection size before and after. Then write the correct version and show it working.

Explain It

A comment covering:

- Why this passes every test you will write for it
- What happens in production, and why the failure does not look like a crash
- **The rule that prevents it**, stated in one sentence

Also print whether assertions were enabled, using the detection trick, so the output explains itself either way.

Part Four: Which Check, Where

Write two methods:

1. One validating a **public caller's argument**
2. One asserting an **internal invariant** your own code just established

Use an exception for the first and `assert` for the second.

Explain It

A comment giving the test you would apply to decide between them. One sentence per branch, and it should be a question you can actually answer at a keyboard.

Acceptance Criteria

- ☐ Runs with `java TestKit.java`, no third-party imports
- ☐ `Suite` supports `test`, `assertEquals` and `assertThrows`

- ☐ `assertThrows` distinguishes "nothing thrown" from "wrong type thrown"
- ☐ A test throwing unexpectedly is a failure, not a crash of the run
- ☐ Parameterised cases are named with their inputs
- ☐ A comment says why `case 3` would be a worse name
- ☐ The summary shows counts, plus the exit code a real harness would use
- ☐ Shared state makes the second check fail; fresh state makes it pass
- ☐ A comment explains why order-dependent tests are worse than none
- ☐ The assert-hidden removal is shown **not** happening without `-ea`
- ☐ The corrected version is shown working
- ☐ The one-sentence rule is stated
- ☐ Exception and assert are each used where they belong, with the deciding test written down

Stretch

Add a timeout to `test`, so a body that hangs is reported as a failure rather than hanging the run.

Then write a comment on what you had to introduce to make that work, and why a timeout is harder to implement than it looks. Consider what happens to the thread running the hung test after you report it.

Hint, if part three shows the removal happening

Check how you ran it. With `-ea` the assert executes and the removal works, which is exactly why this bug survives testing. Run it without the flag, which is how production runs.